

■ CINEMA-APP PWA

GUÍA INTEGRAL DE ARQUITECTURA Y DEFENSA ORAL

Manual de conceptos técnicos, flujo de datos, librerías, seguridad y resolución de dudas puntuales de código para examen final.

Materia:	Programación IV
Profesor:	Manuel Quintana
Alumno:	Santiago Gemignani
Stack:	Angular 19 Standalone, Supabase (PostgreSQL), PWA, Vercel
Despliegue Live:	https://cinema-app-coral.vercel.app
Fecha:	Septiembre 2026

■ Objetivo de esta guía para tu defensa oral:

Esta guía está redactada con explicaciones claras, fundamentos teóricos y analogías de la vida real. Te permitirá responder con total seguridad técnica cualquier pregunta que te haga el profesor sobre el flujo de la aplicación, inyección de dependencias, tipado con TypeScript, reactividad moderna con Signals, manejo de sesiones con JWT y exportación de archivos en tiempo real.

1. Arquitectura General y Flujo de la Información

CinemaApp está construida siguiendo la arquitectura recomendada por Google en **Angular 19**: un modelo desacoplado basado en **Componentes Standalone** (sin NgModules pesados), **Servicios Centralizados Singleton** y **Signals Reactivos** para la gestión del estado.

Capa	Tecnología / Archivo	Responsabilidad
1. Persistencia	Supabase (PostgreSQL)	Almacena tablas relacionales: películas, funciones, salas, compras, entradas, usuarios, auditoría. Aplica Row Level Security (RLS).
2. SDK Conexión	@supabase/supabase-js	Cliente HTTP y WebSocket que envía consultas autenticadas con la anon key y el Bearer Token JWT del usuario.
3. Lógica & DI	Servicios Angular (@Injectable)	PeliculaService, AuthService, AdminService, CarritoService. Encapsulan la lógica de negocio y llamadas a la BD.
4. Reactividad	Signals (signal(), computed())	Estructuras de datos que notifican a la vista únicamente cuando su valor cambia, evitando re-renders innecesarios.
5. Vista UI	Templates HTML + CSS	Directivas modernas de control de flujo (@for, @if) y property bindings ([src], {{ }}).

2. Explicación de Conceptos y Dudas Puntuales

2.1. ¿Cómo funciona @Injectable?

@Injectable es un decorador de Angular que le comunica al compilador que esa clase puede ser administrada por el sistema de **Inyección de Dependencias (Dependency Injection - DI)**.

En nuestro código vemos:

```
@Injectable({ providedIn: 'root' })
```

¿Qué significa providedIn: 'root'? Significa que Angular crea una **única instancia compartida** (Patrón de diseño *Singleton*) para toda la aplicación desde la raíz. Todos los componentes que pidan PeliculaService o AuthService van a compartir exactamente la misma memoria y el mismo estado.

¿Cómo se consume en un componente? Antes se pasaba por el constructor; en Angular 19 se usa la moderna función inject():

```
private peliculaService = inject(PeliculaService);
```

Analogía para la defensa: Es como el sistema de agua corriente de un edificio. En vez de que cada departamento (componente) construya su propio pozo de agua (hacer new Servicio()), el edificio tiene una bomba central (Singleton) y cada departamento simplemente abre la canilla (inyecta la dependencia).

2.2. ¿Qué significa: `import { createClient, SupabaseClient } from '@supabase/supabase-js'`?

Esta línea importa las herramientas oficiales del SDK de Supabase para JavaScript/TypeScript:

- **createClient:** Es la función constructora que inicializa la conexión con el servidor. Recibe dos parámetros fundamentales:

1. *URL del proyecto:* `https://qhpbrgyyilfmvaktkjd.supabase.co` (dirección de tu backend).

2. *Anon Public Key:* Clave criptográfica pública que identifica tu app y valida que las peticiones vayan al proyecto correcto.

Al ejecutarse, `createClient` configura internamente un cliente HTTP con headers predeterminados, administra el token JWT en el `localStorage` del navegador y abre un canal WebSocket para escuchar cambios en tiempo real.

- **SupabaseClient:** No es código ejecutable, es el **Tipo TypeScript**. Sirve para indicarle a TypeScript qué métodos existen en ese cliente (como `.from()`, `.auth`, `.storage`), dándonos autocompletado inteligente y evitando errores tipográficos.

2.3. ¿Cómo funciona el Manejo de Sesión? ¿Dónde se ejecuta y cómo?

El manejo de sesión está centralizado en `src/app/core/services/auth.service.ts`. Funciona mediante **Tokens de Acceso JWT (JSON Web Tokens)** y se divide en 3 momentos clave:

- 1. Dónde se guarda la sesión:** Cuando el usuario ingresa su email y contraseña en `login()`, Supabase valida las credenciales y devuelve una **Sesión** que contiene un `access_token` y un `refresh_token`. El SDK de Supabase guarda automáticamente estos tokens en el `localStorage` del navegador bajo una clave segura.
- 2. Al arrancar o recargar la página (`initAuthSession`):** En el constructor de `AuthService` se ejecuta `this.supabase.auth.getSession()`. Si hay un token válido guardado en el navegador, el usuario **no pierde la sesión al recargar la página**: se recuperan sus datos y se carga su perfil con `loadUserData(userId)`.
- 3. Escucha reactiva en tiempo real (`onAuthStateChange`):** Supabase emite eventos globales cada vez que alguien inicia sesión (`SIGNED_IN`), cierra sesión (`SIGNED_OUT`) o renueva su token expirado (`TOKEN_REFRESHED`). El servicio reacciona actualizando sus Signals:
 - `currentUser.set(session?.user ?? null);`
 - `currentUserData.set(datosPerfil);` (nombre, rol, crédito, puntos).
- 4. Protección de Rutas con Guards:** En `admin.guard.ts` y `auth.guard.ts` se verifica `authService.isAdmin()`. Si alguien intenta entrar a `/admin` sin tener el rol `'admin'`, el Guard intercepta la navegación y lo redirige automáticamente a `/login`.

2.4. ¿Por qué veo tanto la palabra 'interface'? ¿Qué es y para qué sirve?

En TypeScript, una **interface** es un **contrato o molde** que define la estructura obligatoria que debe tener un objeto. **Ejemplo en nuestro proyecto (`pelicula.interface.ts`):**

```
export interface Pelicula { id: number; titulo: string; sinopsis: string; duracion_minutos: number;
imagen_url: string; clasificacion_edad: '+13' | '+18' | 'ATP'; precio_normal: number; }
```

¿Por qué es fundamental para la defensa?

- 1. Type Safety (Seguridad de tipos):** Si en el código escribís por error `pelicula.titulo` o intentás asignarle un texto a `precio_normal`, TypeScript no te deja compilar y te avisa exactamente dónde está el error.
- 2. Cero peso en producción:** Las interfaces **desaparecen** cuando el proyecto se compila a JavaScript normal. No ocupan memoria ni pesan en el navegador; son una herramienta pura de desarrollo para programar sin fallas. *Analogía:* La interface es el plano del arquitecto. El plano no es una casa física, pero garantiza que cuando los albañiles construyan la casa, no se olviden de poner las puertas ni las cañerías.

2.5. ¿Cómo funciona 'export type UserRole'? ¿De dónde lo agarra y cómo sabe qué es?

En `src/app/core/models/usuario.interface.ts` tenemos:

```
export type UserRole = 'cliente' | 'empleado' | 'admin';
```

¿Qué es? Es un **Union Type de literales** en TypeScript.

¿Cómo sabe qué es? Le dice al compilador: *'Una variable de tipo UserRole NO puede ser cualquier texto;*

únicamente puede ser una de estas tres palabras exactas: cliente, empleado o admin'. Si alguien intenta escribir `rol = 'superusuario'`, TypeScript marca error inmediato.

¿De dónde lo agarra? La palabra `export` hace que esté disponible públicamente para todo el proyecto. En `auth.service.ts` o en `role.directive.ts` lo 'agarramos' escribiendo arriba:

```
import { UserRole } from '../models/usuario.interface';
```

2.6. El Ciclo de las Películas: ¿Cómo se conecta todo y cómo se muestran varias si codeo solo una?

Esta es la pregunta reina de tu defensa oral sobre el flujo frontend. El circuito completo funciona así en 4 pasos:

Paso 1: La Base de Datos y el Servicio	En Supabase tenemos la tabla peliculas con 6 registros. En pelicula.service.ts ejecutamos: <pre>const { data } = await this.supabase.from('peliculas').select('*');</pre> Supabase devuelve un Array (lista) de 6 objetos en formato JSON que cumplen con la interface Pelicula[].
Paso 2: El Componente (home.ts)	En home.ts inyectamos el servicio y guardamos esa lista en un Signal: <pre>peliculas = signal<Pelicula[]>([]);</pre> En el ngOnInit(), el servicio llena ese signal. Como es reactivo, la plantilla HTML se entera al instante de que ya llegaron los datos.
Paso 3: El bucle @for en el HTML	¿Por qué parece que codeamos una sola? Porque en home.html usamos la directiva de control de flujo de Angular 19: <pre>@for (pelicula of peliculasFiltradas(); track pelicula.id) { ... }</pre> Ese bloque actúa como un bucle for clásico. Nosotros diseñamos el molde de una sola tarjeta HTML , y Angular se encarga de clonarla y multiplicarla en el DOM tantas veces como películas haya en el Array. • En la 1ª vuelta: la variable temporal pelicula vale Dune 2. • En la 2ª vuelta: pelicula vale Intensa-Mente 2. • En la 3ª vuelta: pelicula vale Deadpool & Wolverine.
Paso 4: Property Binding y Detalle	Dentro de la tarjeta escribimos: <pre></pre> Los corchetes [src] le indican a Angular que evalúe la variable TypeScript y extraiga la propiedad imagen_url de esa película en particular. ¿Y en pelicula-detalle.html? En esa pantalla NO hay bucle porque se visualiza una sola película . El componente lee el ID que viene en la ruta (ej. /pelicula/2), busca en Supabase con .eq('id', 2).single() y le asigna sus datos al objeto individual p.

2.7. ¿Cómo funciona que con un botón creo un PDF o un Excel con todo guardado?

Ambas funciones se ejecutan **100% en el navegador del cliente**, sin necesidad de saturar el servidor:

1. Generación de PDF (jsPDF + autoTable):

- En admin.ts la función descargarReportePDF() instancia el motor gráfico con const doc = new jsPDF();.
- Dibuja títulos vectoriales con doc.setFontSize(18); doc.text('INFORME FINANCIERO', 14, 20);.
- Convierte los datos de ventas y compras en una tabla estética mediante autoTable(doc, { head: [...], body: filas }).
- Al final ejecuta doc.save('reporte_facturacion.pdf');. Esto genera un **Blob (objeto binario)** en la memoria RAM del navegador y dispara la descarga nativa.

2. Generación de Excel / CSV:

- En descargarReporteExcel() no se necesita ninguna librería pesada. Se construye un string en formato **CSV (Valores Separados por Comas)** con saltos de línea \n.
- Se le antepone el carácter especial \uFEFF (BOM UTF-8) para que Microsoft Excel lo abra con tildes y ñes correctas.

- Se envuelve en un `new Blob([contenido], { type: 'text/csv;charset=utf-8;' })`.
- Se crea un enlace virtual en memoria (`document.createElement('a')`), se le asigna `a.href = URL.createObjectURL(blob)` y se simula un clic con `a.click()`. Excel lo abre de forma automática con todas las columnas formateadas.

2.8. ¿Cómo con un botón genero un QR con identidad real?

Un Código QR **no es una simple foto estática**: es una matriz bidimensional que almacena datos alfanuméricos según el estándar internacional ISO/IEC 18004. En CinemaApp lo implementamos así:

1. Construcción del Payload (Identidad Real):

Cuando el usuario finaliza la compra en checkout.ts, generamos un identificador único y un objeto JSON con los datos auditables de la entrada:

```
const payload = JSON.stringify({ idCompra: 124, codigo: 'TICK-8842', pelicula: 'Dune 2', sala: 1, butacas: ['E4', 'E5'], hash: 'f8a92...' });
```

2. Conversión Algorítmica con la librería 'qrcode':

Llamamos a `QRCode.toDataURL(payload, { errorCorrectionLevel: 'M' })`.

El algoritmo matemático convierte ese texto en un patrón geométrico de cuadrados blancos y negros con redundancia contra roturas (corrección de errores Reed-Solomon) y devuelve un string en formato **Base64** (...).

3. Visualización y Validación en el Cine:

- En la app del cliente: esa cadena Base64 se asigna directamente al `[src]="qrCodeUrl()"` de la entrada y se incrusta en el PDF descargable.
- En la puerta de entrada del cine (**Ruta /validador**): El empleado del cine abre la pantalla del validador con la cámara del celular, escanea el código QR, la app decodifica el JSON, consulta en tiempo real a Supabase si esa entrada ya fue utilizada, y si es válida la marca como estado = 'usada'. ¡Esto garantiza que nadie pueda ingresar dos veces con la misma entrada!

3. Tips y Frases Clave para Lucirte en la Defensa con Manuel Quintana

Pregunta Típica del Profesor	Respuesta Técnica Sugerida
'¿Por qué usaron Signals en lugar de RxJS con BehaviorSubjects?'	'Porque Signals es el nuevo estándar reactivo de Angular a partir de la versión 17/19. Ofrece reactividad de grano fino (fine-grained reactivity), lo que significa que el framework actualiza únicamente el nodo del DOM que cambió sin tener que chequear todo el árbol de componentes (Change Detection OnPush), además de simplificar la sintaxis evitando memory leaks por desuscripciones manuales.'
'¿Cómo manejan la seguridad si la anon key de Supabase está pública en el frontend?'	'La seguridad de Supabase no depende de ocultar la anon key, sino de la capa de Row Level Security (RLS) directamente en el motor PostgreSQL. Cada tabla tiene políticas estrictas que validan el token JWT del usuario (auth.uid()), impidiendo que un usuario común modifique precios, cartelera o funciones.'
'¿Por qué la app es una PWA (Progressive Web App)?'	'Porque cuenta con un manifest.webmanifest con iconos adaptativos y un ngsw-worker.js (Service Worker de Angular). Esto le permite al usuario instalar la app en su celular (iPhone o Android) como si fuera nativa, guardando en caché estáticos para cargar al instante incluso con mala conexión.'